

crashes. GC helps prevent this by making sure memory is freed up when objects or data are no longer in use.

Common garbage collection methods are:

Reference counting: Keeps track of how many parts of the program are using a piece of data. When the count drops to zero, the memory can be freed.

Tracing (mark-and-sweep, generational):

- Mark-and-sweep scans memory to mark active objects and then clears the rest.
- Generational GC splits memory into ‘young’ and ‘old’ objects, optimising how often each part is cleaned.

Stop-the-world vs concurrent GC:

- Stop-the-world pauses the entire program during garbage collection.
- Concurrent GC runs alongside the program to reduce pauses and improve responsiveness.

Java’s garbage collection strategy

Java runs on the Java Virtual Machine (JVM), and one of its biggest strengths is the flexibility it offers in memory management. The most used JVM, HotSpot, supports multiple garbage collection algorithms—each tuned for different needs like low latency, high throughput, or minimal pause times.

Modern enterprise applications — whether cloud-based microservices, SaaS platforms, or high-frequency trading systems — require predictable performance, scalability, and stability. Java’s diverse garbage collection strategies offer tailored solutions:

- Low-latency, real-time apps benefit from ZGC and Shenandoah.
- Large-scale services with high throughput leverage G1

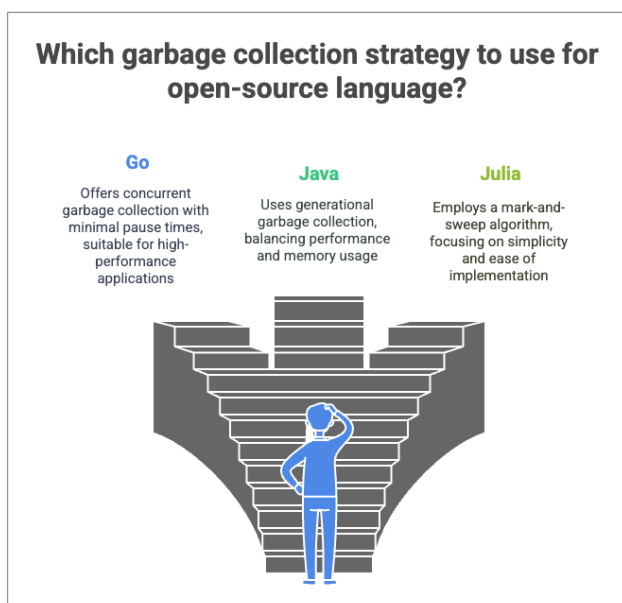


Figure 2: Garbage collection strategies

Java GC: Strengths vs weaknesses

Aspects	Strengths	Weaknesses
GC algorithm options	Multiple choices (Serial, Parallel, CMS, G1, ZGC, Shenandoah) for different needs	Requires knowledge to choose the right GC for your app
Performance tuning	Can optimise for either high throughput or low pause times	May involve trial-and-error tuning and detailed benchmarking
Observability and tooling	Rich tools (JVisualVM, GC logs, Flight Recorder) for monitoring and debugging	Tools require setup and understanding of JVM internals
Scalability	Handles large heaps and multi-core systems efficiently	GC tuning may become more complex as system scale increases

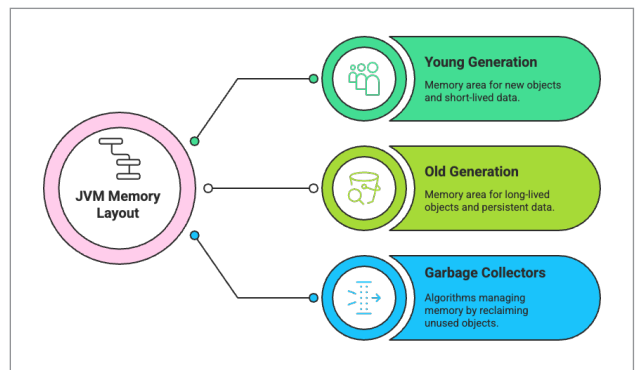


Figure 3: JVM memory layout

and parallel GC.

- Small, embedded apps can still efficiently run on serial GC.

Java’s GC makes it ideal for:

- Enterprise backends
- Large-scale web services
- Android applications
- High-performance financial systems

Go’s garbage collection strategy

In Go, garbage collection automatically reclaims memory that is no longer reachable or needed by the program. When a Go program creates objects in memory (allocated on the heap), these objects may eventually become unreachable. Go’s garbage collector identifies such objects and frees up